Vis Comput (2015) 31:93–104
DOI 10.1007/s00371-013-0913-2

ORIGINAL ARTICLE

An image-space algorithm for immersive views in 3-manifolds and orbifolds

Pierre Berger · Alex Laier · Luiz Velho

Published online: 18 January 2014

Abstract We describe the first image-space ray-tracing algorithm to generate interior views of flat or hyperbolic 3-manifolds, orbifolds, and similar polyhedral complexes. The complexity of our algorithm is linear in the number of echoes for a given number of polygons, compared to exponential complexity for the object-space algorithm chosen as standard.

Keywords 3-Manifolds · Hyperbolic geometry

1 Introduction

With the Geometry Center [16], William Thurston initiated a program to disseminate modern geometry using interactive visualization. This initiative resulted in software that provided immersive views of 3-manifolds or 3-orbifolds. The goal of such visualization algorithms is to render what one would see if he/she lived inside that space. In particular, hyperbolic manifolds, which will be formally defined further, are fundamental: according to Thurston [15], most of the manifolds are hyperbolic—i.e., there is a very rich variety of them.

In a view of a compact manifold, a single object appears many times, and in general its images fill up all the horizon. Indeed, a ray of light might loop and such paths can represent every element of the manifold's fundamental group. In hyperbolic manifolds, the sizes of the object's copies fall exponentially w.r.t. their distance to the camera, whereas the number of such copies grows exponentially (since the fundamental group grows exponentially fast).

We present here a new image-space visualization algorithm, based on ray tracing, to render immersive views in flat or hyperbolic manifolds and orbifolds. This algorithm is also generalizable to other geometries as discussed in Sect. 8.

The proposed algorithm is much more efficient than the previous algorithms based on object-space visualization. For typical example, it allows interactive visualization of high-resolution images of scenes with full optical complexity, and can render a scene formed by an arbitrarily large number of copies of the same object.[1] Amazingly, although this goal may seem unreachable, such a great number of objects reach subpixel precision and change dramatically the pattern of the landscape with respect to the one formed by what is currently achievable with object-space methods. (See Fig. 3.)

One of the motivations of our work was to develop state-of-the-art software to generate immersive visualizations for the exhibit *"Regards dans les espaces de dimension 3"*. This scientific exhibition aims to disseminate the notion of 3D spaces to a large audience. In particular, it presents through videos and interactive installations the concepts behind the Thurston–Perelman Geometrization theorem. The event took place in March 2013 at the Universite Paris13-Villetaneuse and will tour around the world. Figure 6 shows some results.

P. Berger
CNRS-LAGA Universite Paris 13, Villetaneuse, France
e-mail: berger@math.univ-paris13.fr

A. Laier
Universiade Federal Fluminense, Niterói, Brazil
e-mail: alex.laier@gmail.com

L. Velho (✉)
Instituto de Matematica Pura e Aplicada, Rio de Janeiro, Brazil
e-mail: lvelho@impa.br

[1] In comparison with previous work, our algorithm can render ray paths 25 times deeper than the ones in object-space algorithms (see Sect. 7). This is a significant improvement even if we consider the recent development in graphics hardware.

2 Related work

As previously mentioned, the seminal work in mathematics visualization was developed at the Geometry Center, during the period of 1994–1998. For that purpose, the main software platform of the center was Geomview, an interactive 3D graphics viewer [1]. This software featured a powerful plugin architecture, which allowed it to be extended through new modules, to a variety of uses ranging from animation to video production and scientific simulation.

Geomview was based on a generic object-oriented graphics library, OOGL, which supported by design viewing in Euclidean, elliptical and hyperbolic geometries exploiting the fact that the graphics pipeline using projective transformations allowed an elegant unification of the modeling and camera transformations for these three geometries.

Geomview was initially developed for the Silicon Graphics workstation using the GL interactive low-level graphics. It also allowed output in the Renderman format for off-line high-quality rendering. Subsequently, Geomview was ported to the X11 Linux platform using OpenGL and is still in use today by many practitioners and researchers, even after the termination of the Geometry Center activities in 1999.

The plugin architecture of Geomview, made possible, among other things the development of *Maniview*, a module that works as a Manifold Viewer [9]. Maniview implements the discrete group structure to generate the elements of $PGL(\mathbb{R}, 4)$ required to allow the visualization of the insider's view of a manifold.

Maniview has also been used in conjunction with *Snappea*, a program developed at the Geometry Center by Jeffrey Weeks for computing hyperbolic structures on three-dimensional manifolds [19]. Other software for modeling and investigation in 3-manifolds includes Regina [4], implemented in C++ and used in various related projects.

Subsequently, Weeks created a software for real-time rendering of curved spaces using OpenGL graphics [20]. It is a standalone program that exploits features of the programmable graphics pipeline, such as shaders and textures.

Similarly to Geomview, jReality is a Java-based, 3D scene graph package designed for mathematical visualization at TU-Berlin and associated research centers [23]. Among other features, JReality has intrinsic support for Euclidean, hyperbolic and elliptical geometries, which can be used for creating immersive views of 3-manifolds. The software uses JOGL as a backend for interactive OpenGL rendering.

One of the main goals of using computer graphics methods for mathematics visualization is to provide insights into the world of geometric structures, both for research and education purposes [11]. In that respect, the Geometry Center produced highly successful videos, such as "Not-Knot" [8] and "The Shape of Space" [18], which were complemented by illustrated books [22].

At another level, virtual reality installations allows the user, not only to have a glimpse at the visual landscape inside a 3-manifold, but also to experience the sensation of being completely immersed in such an enviroment. Two projects following this directions are Mathenautics [12] and Alice [6].

Charles Gunn, who worked at the Geometry Center in the pioneer group that created OOGL/Geomview, and later on at TU-Berlin as one of the main developers of jReality, proposed in 2010 a framework for metric-neutral visualization that greatly simplifies the rendering of non-Euclidean geometries [7]. In particular, this paper describes several advances that have an impact on the architecture of generic virtual reality systems, for example, a metric-neutral algorithm for head tracking in VR for the different metric spaces of interest.

In addition to direct visualization and interaction, the properties of non-Euclidean geometric structures can be exploited to investigate other types of data, such as the World Wide Web connections [13].

A common characteristic of all visualization software developed in previous work for creating immersive views of 3-manifolds is that they employ object-space rendering methods. This fact, in practice, imposes a limit on the scene complexity that can be depicted by these programs. In contrast, our new visualization program uses an image-space algorithm that can efficiently render scenes with several orders of more complexity magnitude than possible in previous work. For a comparison, see Fig. 3.

Finally, although not directed at 3D manifolds, an image-space rendering method called *reverse pixel lookup*, described in [17], has some connections to the image-space visualization algorithm proposed by us, in the sense that it follows the action of a transformation group to render the image. The goal of the method is to produce a two-dimensional ornament in a tesselation of the hyperbolic plane. This algorithm renders a tiling of the Poincare disk by recursively mapping back pixels of a tile according to the group action to a central domain to find the corresponding pattern color for that pixel.

3 Three-manifolds and orbifolds

A *3-manifold* is a paracompact, metric space locally modeled by *charts* on open sets of $\mathbb{R}^3$. The coordinate changes are diffeomophisms of open sets of $\mathbb{R}^3$. The unit sphere of $\mathbb{R}^4$ and the torus $\mathbb{R}^3/\mathbb{Z}^3$ are examples of 3-manifolds.

A *3-orbifold* is a paracompact, metric space locally modeled by *charts* on open sets of quotients of $\mathbb{R}^3$ by finite groups. The coordinate changes are diffeomophisms of open sets of these quotients.

For instance, if the finite group acts by a single symmetry with respect to a plane, then the quotient is $\mathbb{R}^2 \times \mathbb{R}_+$. If it is the unique action involved (with the trivial one), then the orbifold is a manifold with boundary. If the action of the

finite group is done by symmetries with respect to a plane in a general position, then the group is $(\mathbb{Z}/2\mathbb{Z})^3$ and the quotient is diffeomorphic to $\mathbb{R}^3_+$. If it is the only action involved, then the orbifold is a manifold with corners. Other kinds are possible, for example see [3] or [15].

We remark that the metric of the manifold must be locally equivalent (via charts) to the Euclidean metric on $\mathbb{R}^3$, and likewise for the orbifolds with their corresponding quotients of $\mathbb{R}^3$.

Orbifolds are canonically stratified. An orbifold is a manifold if its stratification is trivial, that is, it has no singularity.

A parametrized curve in a manifold is *differentiable* if its composition with any chart is differentiable (on its domain of definition). A parametrized curve in an orbifold M is *differentiable* if its composition with the chart is the projection of a differentiable curve of $\mathbb{R}^3$ into a quotient of $\mathbb{R}^3$.

A *geodesic* is a smooth curve γ between two points such that given $t_1 < t_2 < t_3$ close to each other:

$$d(\gamma(t_1), \gamma(t_3)) = d(\gamma(t_1), \gamma(t_2)) + d(\gamma(t_2), \gamma(t_3)).$$

Geodesics are the paths taken by rays of light. From a unit vector at a point of the manifold, there is a unique geodesic in that direction for all the examples that we will give (since their distance is Riemannian).

These definitions are crucial since they define the immersive view in a manifold. Given an object embedded in a manifold, a viewer in the manifold will see in a given direction u, the first intersection between the object and the half geodesic starting at the viewer position and in the direction u. There are three important categories of (Riemannian) manifolds, namely: flat, hyperbolic or elliptical.

Example 1 (Euclidean space $\mathbb{E}^3$) Let N be the Euclidean norm:

$$N(x_1, x_2, x_3) = \sqrt{x_1^2 + x_2^2 + x_3^2}$$

on $\mathbb{R}^3$. The *3-Euclidean space* $\mathbb{E}^3$ is $\mathbb{R}^3$, with metric on $\mathbb{E}^3$ equal to $d_{\mathbb{E}^3}(\underline{x}, \underline{x}') := N(\underline{x} - \underline{x}')$.

The group of isometries is the semi-product $O(3) \ltimes \mathbb{R}^3$ of the orthogonal group and translation groups of $\mathbb{E}^3$.

Example 2 (Hyperboloid model $\mathbb{H}^3$ of the hyperbolic space) Let $q : \mathbb{R}^4 \to \mathbb{R}$ be the (Lorentzian) quadratic form defined by $q(x_1, x_2, x_3, x_0) = x_0^2 - x_1^2 - x_2^2 - x_3^2$. The *3-hyperbolic space* $\mathbb{H}^3$ is

$$\{\underline{x} \in \mathbb{R}^4 : \ q(\underline{x}) = 1\} \cap \mathbb{R}^3 \times \mathbb{R}^+$$

endowed with the metric $d_{\mathbb{H}^3}(\underline{x}, \underline{x}') := \operatorname{arcosh} B(\underline{x}, \underline{x}')$, with B the bilinear symmetric form associated with q: $B(\underline{x}, \underline{x}') = x_0 \cdot x_0' - x_1 \cdot x_1' - x_2 \cdot x_2' - x_3 \cdot x_3'$, where $\underline{x} = (x_i)_i$ and $\underline{x}' = (x_i')_i$.

The geodesics $\mathbb{H}^3$ cannot be straight, since $\mathbb{H}^3$ is not an affine subspace of $\mathbb{R}^4$. However, the geodesic passing trough $x \in \mathbb{H}^3$ in the direction u tangent to $\mathbb{H}^3$ at x is the intersection of $\mathbb{H}^3$ with the 2-plane spanned by the vectors $\mathbf{0x}$ and $\mathbf{u}$ of $\mathbb{R}^4$.

The group of isometries of $\mathbb{H}^3$ is equal to the group $O(3, 1)$ of 4×4-matrices which leaves invariant the form q. Every matrix of this group acts on $\mathbb{R}^4$ by matrix product and leaves invariant $\mathbb{H}^3$. By definition of g, every matrix of $O(3, 1)$ acts as an isometry of $\mathbb{H}^3$.

Example 3 (3-Sphere $\mathbb{S}^3$) Let $Q : \mathbb{R}^4 \to \mathbb{R}$ be the Euclidean quadratic form defined by $Q(x_1, x_2, x_3, x_0) = x_0^2 + x_1^2 + x_2^2 + x_3^2$. The *3-sphere* $\mathbb{S}^3$ is

$$\{\underline{x} \in \mathbb{R}^4 : \ Q(\underline{x}) = 1\}$$

endowed with the metric $d_{\mathbb{S}^3}(\underline{x}, \underline{x}') := \operatorname{arcos} \langle \underline{x}, \underline{x}' \rangle$, with $\langle \underline{x}, \underline{x}' \rangle$ the Euclidean inner product: $x_0 \cdot x_0' + x_1 \cdot x_1' + x_2 \cdot x_2' + x_3 \cdot x_3'$, where $\underline{x} = (x_i)_i$ and $\underline{x}' = (x_i')_i$.

The geodesic from $x \in \mathbb{S}^3$ in the direction u tangent to $\mathbb{S}^3$ at x is the intersection of $\mathbb{S}^3$ with the 2-plane spanned by the vectors x and u of $\mathbb{R}^4$.

The group of isometries of $\mathbb{S}^3$ is equal to the orthogonal group $O(4)$ of 4×4-matrices which leaves invariant the norm N.

A fundamental way to construct 3-orbifolds (and manifolds in particular) is to take a discrete subgroup Γ of $O(3) \ltimes \mathbb{R}^3$ (resp. $O(3, 1)$, resp. $O(4)$) and then regard the orbit space $M := \Gamma \backslash \mathbb{E}^3$ (resp. $\Gamma \backslash \mathbb{H}^3$, $\Gamma \backslash \mathbb{S}^3$)) of the Γ-action. It is easy to show that M is an orbifold (whenever the action is proper). Such orbifolds are called *developable* and *flat* (resp. *hyperbolic, elliptical*).

Example 4 The (flat) 3-torus is the quotient $\mathbb{Z}^3 \backslash \mathbb{E}^3$ where the (free) action of $\mathbb{Z}^3$ on $\mathbb{E}^3$ is done by translation.

Example 5 The 3-projective space $\mathbb{P}^3$ is the quotient $\{\pm id\} \backslash \mathbb{S}^3$. It is an elliptical manifold. It is also equal to the quotient of $\mathbb{R}^4 \backslash \{0\}$ by the action of $\mathbb{R}$ by homotopy. We denote by $(z_1 : z_2 : z_3 : z_4) \in \mathbb{P}^3$ the image of $(z_1, z_2, z_3, z_4) \in \mathbb{R}^4 \backslash \{0\}$.

In Sect. 7 we will recall the scheme of all previous algorithms for rendering flat and hyperbolic orbifolds. They correspond to object-space algorithms.

Here, we introduce a new rendering algorithm for interior views in the flat, hyperbolic or elliptical orbifold. It is an image-space algorithm. Also, we can apply it to non-developable orbifold (see Prop. 6 of [10]) and even to a more general class of spaces called polyhedral complexes.

4 Visualization of three-dimensional spaces

A 3D visualization algorithm renders an image of a three-dimensional scene according to a view specification. The input of the algorithm is a scene description composed of: an ambient 3D space; 3D shapes placed in this ambient

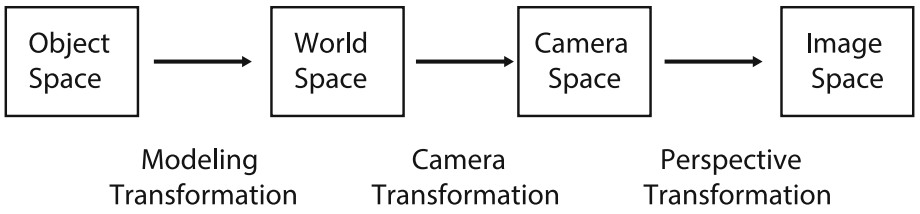


Fig. 1 Viewing Transformation Pipeline

space and a viewpoint, among other parameters. The output is a two-dimensional view. In that sense, the rendering process transforms geometric three-dimensional information into visual two-dimensional information.

4.1 The Viewing Transformation Pipeline

To understand better this process, let us recall the *viewing transformation pipeline*, which relates the different spaces and coordinate systems involved in the computation of a rendered image.

Each shape of an object $o \in S$ is naturally defined in its own local *object coordinate system*. All the objects are then placed in the global *world coordinate system* of the scene S by a modeling transformation that embeds the object into the ambient space (which can be a manifold or orbifold). The view is specified by a viewing transformation V, which defines the *camera coordinate system* relative to the world (e.g., its position and orientation). Finally, the objects that are visible from the camera are mapped to the *image coordinate system* (which implements the viewing window). This pipeline is shown in Fig. 1.

The transformations of the viewing pipeline can be defined by real projective mappings in homogeneous coordinates in $\mathbb{P}^3$. This scheme has been adopted as a standard in most graphics systems because it unifies all the transformations involved using 4×4 projective matrices. In that way, the 3D objects are immersed in projective space, transformed and then projected to a 2D image space.

This particular way in which the viewing pipeline is defined opens up the possibility to render views of ambient spaces different from the Euclidean space $\mathbb{E}^3$, in particular, with a flat or hyperbolic geometry, as it will be seen below.

4.2 Types of Algorithms

There are two main types of 3D visualization algorithms. They can be classified into:

- object space;
- image space.

Object-space algorithms apply the direct viewing transformation to points of the objects, while image-space algorithms apply the inverse viewing transformation to rays originating from the camera and corresponding to image pixels.

The structure of these two types of algorithms is described by the pseudo-codes below:

Algorithm 1 Object-space visualization

```
for each o ∈ S do
    Map o from scene to camera space
    if o is visible then
        Project o to image space
    end if
end for
```

The object-space algorithm is widely adopted in computer graphics and is the one used in the OpenGL standard.

Algorithm 2 Image-space visualization

```
for each pixel p ∈ I do
    Generate a ray r in camera space
    Transform r to scene space
    Find the intersection i(r) with visible object o ∈ S
    if i(r) ≠ ∅ then
        Paint pixel
    end if
end for
```

The image-space algorithm is the basis of *ray tracing* rendering methods.

Note that object-space algorithms work at geometric precision and, in principle, must perform full evaluation when transforming objects in the scene, while image-space algorithms work at image resolution and may perform a lazy evaluation of transformations required by each ray.

Furthermore, the opposite nature of these two algorithms has a determinant impact on the complexity of the visualization process and on the strategies used to make them more efficient. This depends on various factors, such as scene and depth complexity, among others.

The bottom line is that rendering acceleration methods exploit some kind of coherence in different classes of scenes. We will return to this point further in the paper, regarding the immersive views of 3-manifolds.

4.3 Inside views in non-Euclidean spaces

With a few exceptions, mentioned in Sect. 2, most 3D visualization softwares support only rendering of scenes in the Euclidean space $\mathbb{E}^3$. However, exploiting the fact that the visualization process resorts to projective transformations to create images of 3D spaces, it is possible to render different

model geometries without structural changes to the visualization algorithm, by using the appropriate transformations in the viewing pipeline.

Thus, besides the Euclidean space $\mathbb{E}^3$, other isotropic model geometries, such as elliptical and hyperbolic, can be rendered by the above visualization algorithms. The specialization required in the pipeline amounts to taking care of the correct transformations that respect the intrinsic metric of the model geometry, as discussed in the previous section.

The isometries for a given space (i.e., flat, hyperbolic or elliptical) define the set of model transformations in the viewing pipeline. They are combined with the camera and perspective transformations, which are defined by the view specification.

These transformations are subsumed as elements of $PGL(\mathbb{R}, 4)$, the projective general linear group, and represented as 4×4 transformations matrices in the visualization algorithm (see [9,14]).

To render inside views of three-dimensional spaces, we generate images that would be seen by an observer (e.g., a camera) placed in that space. Light paths follow geodesics and reveal the topology of the space.

4.4 An insider visualization algorithm for non-Euclidean orbifolds and manifolds

Now, we give an intuitive description of an image-space algorithm for generating inside views of three-dimensional orbifolds and manifolds. We start by constructing a representation of the 3-torus that is suitable for intrinsic ray tracing.

A simple way to construct a 2-torus is to glue pairwise the opposite edges of a filled square. The same applies to the construction of a 3-torus: we merely glue pairwise the opposite faces of a filled cube.

An observer living in this 3-torus, looking above would see himself from below. Similarly, looking toward the right he would see himself from the left, and so on for other directions.

The algorithm below is an image-space algorithm to view an object inside this 3-torus from a camera placed at the observer's position.

(Ray tracing inside a 3-torus)

```
for each pixel p ∈ I do
    Generate a ray r from the camera
    for i ≤ level do
        Find the first intersection i(r) with object
        if i(r) ≠ ∅ then
            Paint pixel break
        else The ray r intersects a face F of the cube
            Translate r so that it continues its path
            from the opposite face of the cube.
        end if
    end for
end for
```

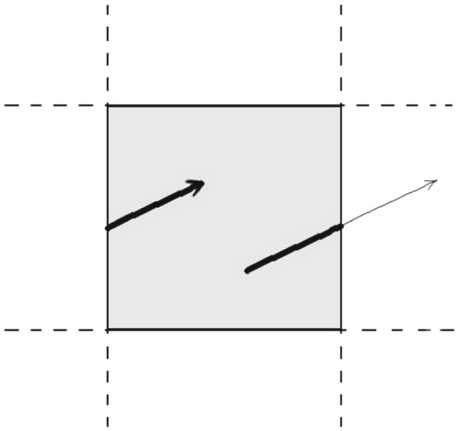


Fig. 2 Path of a ray as it travels through the 3-torus

Figure 2 illustrates the algorithm showing the path of a ray as it leaves and enters the cube (2D view).

In the next section we make formal this intuitive description to consider the general case of 3D orbifolds.

5 Constructions of three-dimensional spaces by gluing faces of a polyhedron

To generalize the mathematical construction of the 3-torus in Sect. 4.4, we create a geometric representation of orbifolds/manifolds based on polyhedrons, in which the corresponding faces are glued. This representation will allow us to generate inside views of these spaces using ray tracing and to present the concepts necessary for specifying the new algorithm.

5.1 Polyhedron

Let us recall a few classical definitions on which polyhedral complexes are based.

A surface S of a manifold G is *totally geodesic* if every geodesic of G tangent to S is included in it. A compact subset C of G is *convex* if every pair of points in C can be joined by a geodesic segment included in C. A *polygon of* G is a convex topological disk included in a totally geodesic surface, whose boundary is formed by a finite union of geodesic segments. The geodesic segments are the *edges* of the polygon and their end points are the *vertices*.

A *polyhedron* Δ is a disjoint union of convex topological closed balls embedded in G. The boundary of Δ is a finite union of polygons $(P_i)_{i \in I_D}$ with disjoint interiors, such that every non-empty intersection $P_i \cap P_j$ is equal to a common edge or a vertex of both P_i and P_j.

An *Euclidean or flat polyhedron* is a G-polyhedron with $G = \mathbb{E}^3$. A *hyperbolic polyhedron* is a G-polyhedron with $G = \mathbb{H}^3$. For instance, every Platonic polyhedron (cube,

dodecahedron,...) is an Euclidean polyhedron ($G = \mathbb{E}^3$). Also a subset of $\mathbb{H}^3$ is a hyperbolic polyhedron iff its image by the following map is a Euclidean polyhedron:

$$\mathbb{H}^3 \ni (x_1, x_2, x_3, x_0) \mapsto \left(\frac{x_1}{x_0}, \frac{x_2}{x_0}, \frac{x_3}{x_0}\right) \in \mathbb{R}^3.$$

The orientation of $\partial\Delta$ (toward the exterior) induces an orientation on each face of Δ.

Definition 1 A G-polyhedral complex is:

- a polyhedron Δ of a manifold G,
- an involution σ of the set of face indices I of Δ,
- an isometry g_i of G which sends P_i onto $P_{\sigma(i)}$, reverse the orientation, and such that $g_i^{-1} = g_{\sigma(i)}$, for every $i \in I$.

such that the relation

$$x \sim y \quad \text{iff } (x = y) \text{ or } (\exists i \in I : x \in P_i \text{ and } g_i(x) = y)$$

spans an equivalence relation on Δ. The equivalence classes form a topological space $S := \Delta/\sim$ called a *G-polyhedral complex.*

For most of the applications, Δ will be connected and so convex.

5.2 Fundamental Domain and Universal Covering

Let Δ be a G-connected polyhedron with isometries $\{g_i\}_{i\in I}$ of G which "glue" the faces of Δ.

Let Γ be the group spanned by the isometries $\{g_i\}_{i\in I}$:

$$\Gamma = \{g_{i_n} \times \cdots \times g_{i_1} : (i_j)_j \in I^{(\mathbb{N})}\}.$$

Suppose the two following conditions:

(a) the manifold G is diffeomorphic to $\mathbb{R}^3$ (and so $\mathbb{H}^3$) or $\mathbb{S}^3$,
(b) $\forall g \in \Gamma, \quad \forall x \in int(\Delta), \quad g(x) = x \Leftrightarrow g = identity$, where $int(\Delta)$ denotes the interior of the polyhedron Δ.

A stronger condition than (b) is the following:

(b') $\forall g \in \Gamma, \quad \forall x \in \Delta, \quad g(x) = x \Leftrightarrow g = identity.$

The polyhedron Δ is called *a fundamental domain*, Γ is called the *fundamental group* (of the orbifold) and G is called the *universal covering* (of the orbifold).

By condition (b), the equivalence relation $\sim$ on Δ can be extended to G by the following for $x, y \in G$:

$$x \sim y \Leftrightarrow \exists g \in \Gamma : y = g(x).$$

Also, the polyhedral complex $\Delta/\sim$ is equal to the space of equivalence classes $G/\sim$. The latter space is usually denoted by $\Gamma\backslash G$.

The following is well known:

Proposition 2 *If conditions* (a) *and* (b) *hold, the space* $\Gamma\backslash M$ *is an orbifold. If conditions* (a) *and* (b') *hold, the space* $\Gamma\backslash M$ *is a manifold.*

Example 6 For instance, the 3-torus has a structure of Euclidean polyhedral complex, where $\Delta = [0, 1]^3$, and the isometries are the translations $(0, 0, \pm 1)$, $(0, \pm 1, 0)$ and $(\pm 1, 0, 0)$.

We can obtain other spaces by changing the translations $(0, 0, \pm 1)$ by composing them with the symmetry about the axis $\{0\}^2 \times \mathbb{R}$.

Example 7 Let us consider a regular dodecahedron of the Euclidean space centered on the origin. For a certain diameter of it, it is included in the unit ball, and its preimage by p is a regular dodecahedron of the hyperbolic space, such that all the dihedral angles between its faces are $\pi/2$. We can glue each face to itself by the identity. The hyperbolic polyhedral complex obtained is an orbifold, and more precisely it is a manifold with corner, called *hyperbolic mirrored dodecahedron*.

Conversely it holds:

Proposition 3 *For every flat (resp. hyperbolic, resp. elliptical) developable orbifold* $\Gamma\backslash M$ *there exists a fundamental domain* $\Delta \subset M$ *and isometries* $(g_i)_i$ *in* Γ *so that* $\Gamma\backslash M$ *is isometric to* $\Delta/\sim$.

One can see that the images of the fundamental domains $(g(\Delta))_{g\in\Gamma}$ tile the universal covering G:

$$\bigcup_{g\in\Gamma} g(\Delta) = M$$

From condition (b), the union $\cup_{g\in\Gamma} g(int(\Delta))$ is disjoint.

Hence, polyhedral complexes are spaces which generalize developable orbifolds. Actually, they generalize even orbifolds. We will see in the Appendix that they include spaces which are not orbifolds, but with physical importance.

6 Image-space algorithm for immersive views in 3-non Euclidean spaces.

We are now ready to describe the new image-space algorithm for visualization in 3-orbifolds M which are flat or hyperbolic.

Using the definitions from the previous section, the algorithm requires a (convex) polyhedral fundamental domain for a given discrete group and gluing identifications among the faces describing how the group acts on these faces.

6.1 Ray tracing in flat and hyperbolic polyhedral complexes

Let $M = \Delta/\sim$ be a flat and hyperbolic polyhedral complex.

Let $O_0 \subset \Delta$ be the preimage of O by $\pi : \Delta \to M$.

We recall that g_i that sends F_i onto $F_{\sigma(i)}$ is an isometry of G which pushes forward the orientation of F_i to the opposite orientation of $F_{\sigma(i)}$.

A *ray* R from a point $c \in \Delta$ in the direction $u \in \mathbb{R}^3\backslash\{0\}$ is the half geodesic from c in the direction u. To trace rays from the camera, each pixel of the screen is canonically associated with a ray R, such that c is the center of projection and u is the view direction for the pixel.

There are two possibilities:

(i) If R intersects O_0, then the pixel color is computed from the intersection of R with O_0.

(ii) If R does not intersect O_0, then we compute the intersection point x of R with the boundary of Δ. Let $\breve{R}$ be the ray included in R and starting at x. The case where x belongs to two different faces F_i is of probability 0 and so is not considered. In other words, we assume that x belongs to a unique face F_i. We continue the ray path using the new ray $R' = g_i(\breve{R})$ starting at point $c' := g_i(x)$.

Note that when R does not intersect O_0, the above procedure is repeated with R' instead of R, until a maximum number of echoes are reached, and then a background color is painted.

6.1.1 *Effective computation of intersection from neutral metric*

We use here the methodology from [7] where details can be found.

Both the Euclidean space $\mathbb{R}^3$ and the hyperbolic space $\mathbb{H}^3$ can be embedded into the projective space thanks to the respective two maps:

$$\mathbb{R}^3 \ni (x_1, x_2, x_3) \mapsto (x_1 : x_2 : x_3 : 1) \in \mathbb{P}^3$$
$$\mathbb{H}^3 \ni (x_1, x_2, x_3, x_0) \mapsto (x_1 : x_2 : x_3 : x_0) \in \mathbb{P}^3$$

The image of the latter maps is the celebrated *Klein model* $\mathbb{K}^3 \subset \mathbb{P}^3$ of the hyperbolic space. Each of the two latter maps sends geodesics, of respectively $\mathbb{R}^3$ and $\mathbb{H}^3$, to intervals of projective lines of $\mathbb{P}^3$. Consequently, it sends the polyhedron Δ to a projective polyhedron $\tilde{\Delta}$. Also, the isometries of $\mathbb{R}^3$ and $\mathbb{H}^3$ are the projective traces of linear transformations of $\mathbb{R}^4$. Consequently, the use of Plücker coordinates for projective lines enables us to compute efficiently the intersections of the ray with the faces of Δ, the intersections of the ray with the object, and the images of the ray by the isometries. These operations are explained in [7].

Note that to perform the computations we need to work with the image $\tilde{\Delta}$ of the object O_0 in the projective space.

6.1.2 *Effective computation of the association of the pixels of the screen with a ray*

In the Euclidean space, each ray R starts from the focal center $c = (c_1, c_2, c_3) \in \Delta$ to a direction $u = (u_1, u_2, u_3) \in \mathbb{R}^3\backslash\{0\}$ canonically associated with the pixel. In other words, the ray R is $c + \mathbb{R} \cdot u$.

To have the Plücker coordinates of its image in $\mathbb{P}^3$, it is sufficient to compute the projective coordinate of two points in this ray, for instance $(c_1 : c_2 : c_3 : 1)$ and $(c_1 + u_1 : c_2 + u_2 : c_3 + u_3 : 1)$.

In the hyperbolic space, the situation is slightly more complicated: everywhere but at $(0, 0, 0, 1)$, the angles in the hyperboloid model are those given by the Euclidean space $\mathbb{R}^4$. The same occurs in the Klein model, everywhere but at $(0 : 0 : 0 : 1)$ the hyperbolic angles are those given by the projective space $\mathbb{P}^3$. Nevertheless, we can move the scene by an (orientation preserving) isometry of $\mathbb{H}^3$ so that $c = (0, 0, 0, 1)$. Then, each ray R starts from the focal center $c = (0, 0, 0, 1) \in \Delta$ to a direction $u = (u_1, u_2, u_3, 0) \in \mathbb{R}^4\backslash\{0\}$ canonically associated with the pixel. To have its Plücker coordinates of its image in $\mathbb{P}^3$, it is sufficient to compute the projective coordinates of two points in this line, for instance $(0 : 0 : 0 : 1)$ and $(u_1 : u_2 : u_3 : 1)$.

6.2 Pseudo-code of the General Algorithm

In summary, the general image-space algorithm for immersive views in flat or hyperbolic polyhedral complexes is as follows:

Algorithm 3 Image-space view in polyhedral complexes

```
for each pixel p ∈ I do
    Let c := 0 and let u be the direction associated with p
    Generate a ray R from (c, u).
    for i ≤ level do
        Find the intersection i(R) with visible object O_0
        if i(R) ≠ ∅ then
            Paint pixel break
        else The ray R intersects a face F_i of Δ
            (⋆) Compute the new origin c' and ray R'.
        end if
    end for
end for
```

The step $(\star)$ is discussed in Sect. 6.1.1.

We remark that this algorithm has complexity linear with the length of the ray path (asymptotic to $level$), that is the number of echoes.

6.3 Examples

In this subsection we show some examples of images produced by the algorithm.

The scene to be rendered consists of a set of objects along with their position and orientation in space.

The algorithm uses ray tracing and enables us to represent analytical surfaces easily and perform computations in closed form. The geometry of objects is described in implicit form, since the basic rendering operation is ray–surface intersection. In that respect, as in any traditional ray tracing method, to include a new geometric type in the system, it is necessary to define functions for computing the intersection of a ray with the object.

Figure 3 depicts a scene consisting of a single sphere inside the mirrored hyperbolic dodecahedron. This landscape is rendered with different levels of echoes, respectively 4, 6, 15 and 100, to reveal the level of visual complexity that can be generated by the algorithm.

Figure 4 shows a single ray view in different spaces (3-torus, 3-sphere, mirrored hyperbolic dodecahedron, and a hyperbolic 3-manifold, with high level of echoes, from an animation in real time.

Figure 5 displays the fundamental domain of the mirrored hyperbolic dodecahedron with different levels of echoes: four as the base reference and nine levels as made possible by our algorithm.

Figure 6 contains three images taken from the immersive installation of the exhibit *"Regards dans les espaces de dimension 3"* . They show curves in flat, hyperbolic and elliptic spaces. The images result from real-time interaction of users in the installation. The user draws a 3D curve in space by manipulating a three-dimensional joystick. The resulting path is rendered with transparency and depth effects.

7 Comparison with object-space algorithms for immersive views in 3-orbifolds

To our knowledge all the previous rendering methods for immersive views in non-Euclidean spaces were based on image-space algorithms. To compare the performance of these two approaches we shall recall their principles. As there is a large literature on this subject we will be brief. For more details see [9,14] and [21].

Object-space algorithms were implemented to render immersive views in manifolds or orbifolds which are developable in the geometry G equal to $\mathbb{H}^3$ or $\mathbb{E}^3$. So, we restrict ourselves to this case. Actually, Weeks also implemented this object-space algorithm for the geometries $\mathbb{S}^3$, $\mathbb{S}^2 \times \mathbb{E}^1$, $\mathbb{H}^2 \times \mathbb{E}^1$ and $\widetilde{SL}_2(\mathbb{R})$, but the basic idea is the same and our presented algorithm seems to be generalizable without substantial change.

Let us explain the main difference between our image-space algorithm and the object-space algorithm. Suppose that we want to render an object O embedded in $\Gamma\backslash G$. Let $O_0 := p(\pi^{-1}(O)) \subset \Delta$.

With our image-space algorithm when a ray R gets out of the polyhedron Δ to enter into the polyhedron $g(\Delta)$, we merely continue the ray path to $g^{-1}(R)$.

For the object-space algorithm when a ray R gets out of the polyhedral Δ to enter into the polyhedron $g(\Delta)$, we must make a copy of the object $g(O_0)$ and render it on the screen.

Abstractly, both constructions are the same, but the way they perform the computations is dramatically different. Indeed, the object-space algorithm is based on the idea that the immersive view in the quotient $\Gamma\backslash G$ is the same as the immersive view in G of the object: $\widetilde{O} := \cup_{g\in\Gamma} g(O_0)$. In general, Γ has infinitely many elements.

The first step of an object-space algorithm is to approximate, given $d \geq 0$, the set $\tilde{O}$ by the set:

$$O_d := O_0 \cup \bigcup_{j\leq d,\ (a_i)_i\in\{1,\dots N\}^j} p \circ g_{a_1} \circ \cdots \circ g_{a_j}(O_0).$$

Observe that when d is large, the set O_d is "close" to $\tilde{O}$.

The second step is to project the object O_d to the Euclidean space. This step is trivial when we deal with $G = \mathbb{E}^3$. When $G = \mathbb{H}$, it is the projection

$$(x, y, z, w) \mapsto (x/w, y/w, z/w) \in \mathbb{E}^3.$$

The last step is to project the object $p(\check{O}_d)$ into the screen space. This is then done by a perspective projection.

The operation of duplication and of rendering are projective transformations and represented by 4×4-matrices. Such operations are done very efficiently by the graphics processing unit (GPU) using OpenGL. This algorithm is not only the one used by Geomview, but also the one behind the software *curved space* for real-time immersive visualization.

However, the latter only works in real time with levels up to 4. The reason is the following: for the simplest hyperbolic manifold, N is equal to 12. Therefore, the cardinality of $\{1, \dots N\}^d$, with $d = 4$, is already 20736. This implies a huge number of objects.

Although some optimizations can be made, they do not change the performance significantly. For instance, some elements of $g_{a_1} \times \cdots \times g_{a_j}$ can be equal to the identity even if these elements are not the identity. Therefore, one can simplify the list to remove redundant duplication.

With such optimizations, the level can be pushed only to $d = 6$ for a simple object with the current high-performance computers. The main reason is that the isometry group of hyperbolic spaces has its cardinality which increases exponentially fast. This means that the cardinality of

$$\Gamma_d := \{g_{a_1} \times \cdots \times g_{a_j} :\ (a_i)_i \in \{1, \dots N\}^j,\ \ j \leq d\}$$

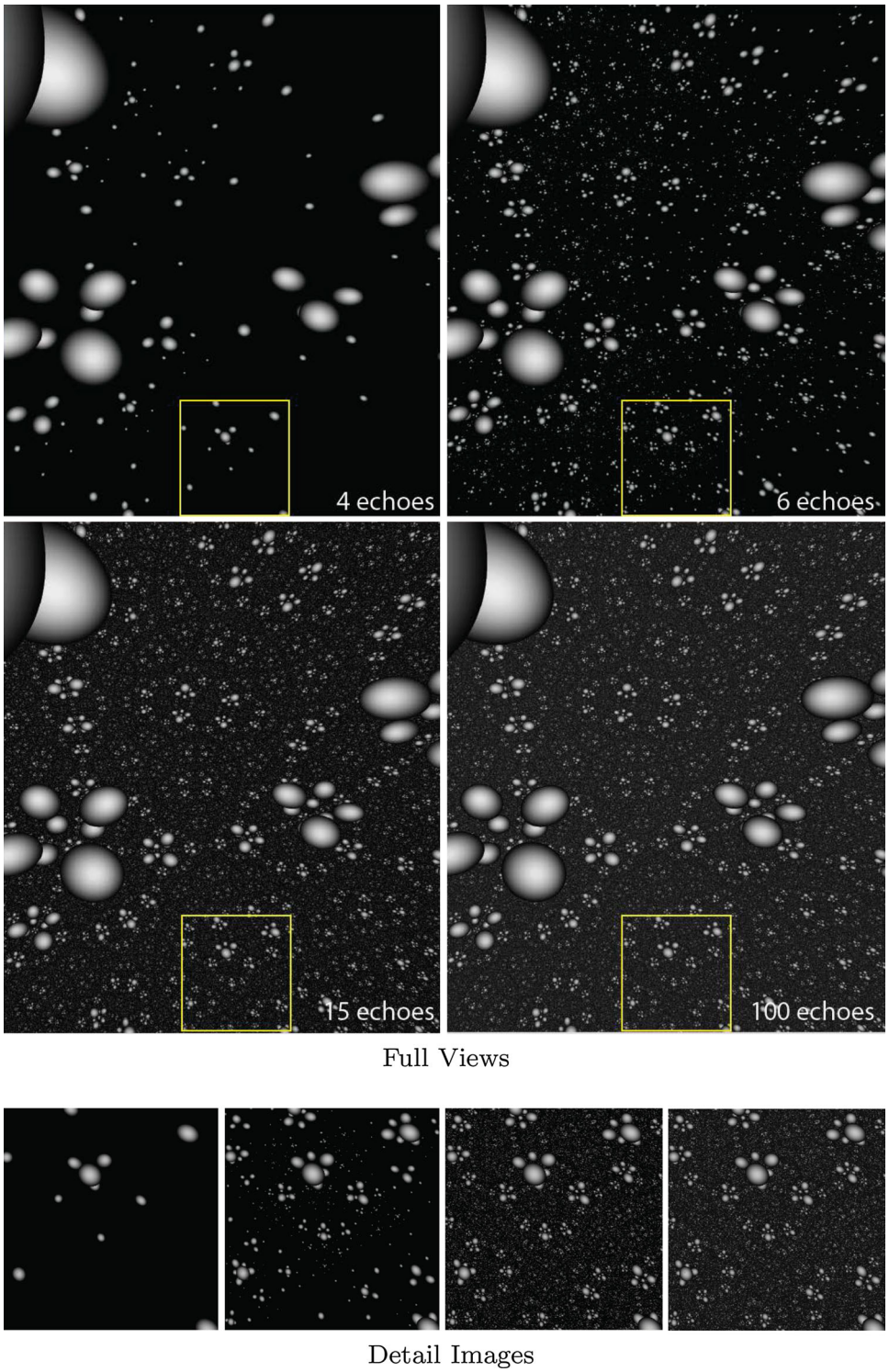


Fig. 3 Landscape of the mirrored hyperbolic dodecahedron rendered with different levels of echoes

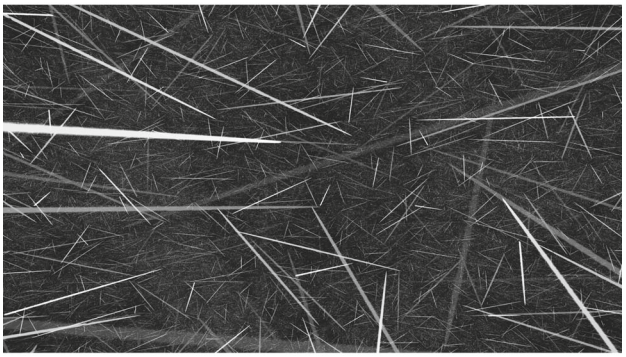
is bounded from below by an exponential function of d. For instance, taking as an example the hyperbolic mirrored dodecahedron (see Sect. 5.2, Example 6), the cardinality of Γ_5 is 128762, the one of Γ_6 is 1276425 and the one of Γ_7 is 12257733, and so on.

As a consequence:

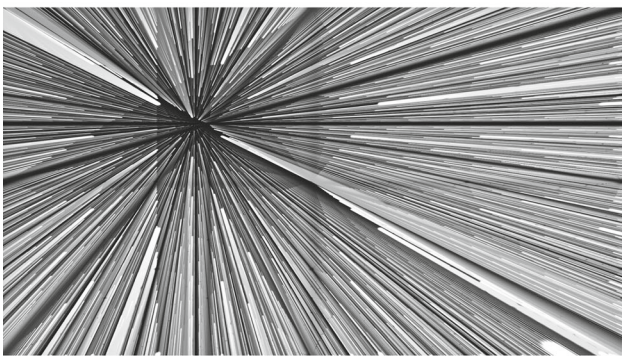
Claim 1 Object-space algorithms are exponential with respect to the depth d of the image.

Moreover, the objects $(g(Ob))_{g\in\Gamma_d}$ tend to be equi-distributed on the horizon, when d goes to infinity. However for low values, such as $d = 6$, the objects $(g(Ob))_{g\in\Gamma_d}$

A ray in the 3-Sphere



A ray in the mirrored hyperbolic dodecahedron



A ray in the 3-Torus

Fig. 4 View showing a single ray in different spaces

are not equi-distributed: the image presents many "holes" although the missing objects are larger than a pixel.

A fundamental problem of object-space algorithms is that they compute many occluded objects which are not needed, because they are not visible. Also, traditional object-space acceleration techniques such as occlusion culling do not help much here, since copies of the object to be drawn need to be generated first. Therefore, attempts to adapt traditional acceleration methods, or even develop new ones, arguably will not be very effective in the case of object-space rendering for immersive views of 3-manifolds and orbifolds. In most cases, the limited efficiency gains will not justify the complexity of implementing the method. Perhaps, this is the

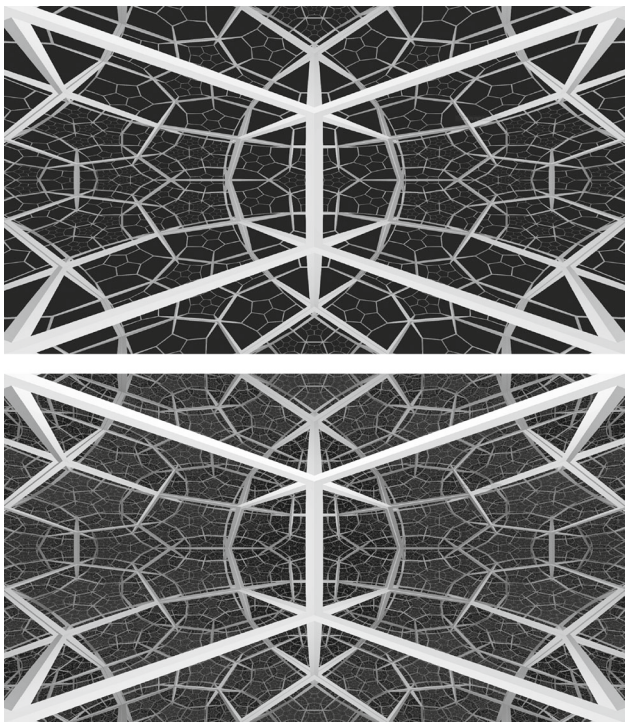


Fig. 5 View showing the fundamental domain of the mirrored hyperbolic dodecahedron with four levels of echoes as the base reference and nine levels as made possible by our software

reason that most existing algorithms rely mainly on the GPU acceleration of OpenGL.

On the other hand, our image-space algorithm for inside views of 3D orbifolds takes full advantage of the classical ray tracing acceleration method based on space subdivision (see [2]). This method is naturally incorporated into the algorithm and does not require additional memory. Furthermore, it can take full advantage of modern programmable GPUs.

8 Conclusions and future work

In this paper we introduced a new image-space algorithm for immersive visualization of flat and hyperbolic 3-manifolds and orbifolds. The algorithm is based on ray tracing and can efficiently render inside views of flat and hyperbolic polyhedral complexes.

Our work improves the state of the art in two main aspects. First, we can generate views of scenes several orders of magnitude more complex than that are possible with previous methods. Second, we can handle a more general type of spaces that could not be visualized up to now (see the "Appendix").

The quality of the image is also improved by performing stochastic anti-aliasing, i.e., we send randomly rays associated with each pixel and then average the different colors obtained.

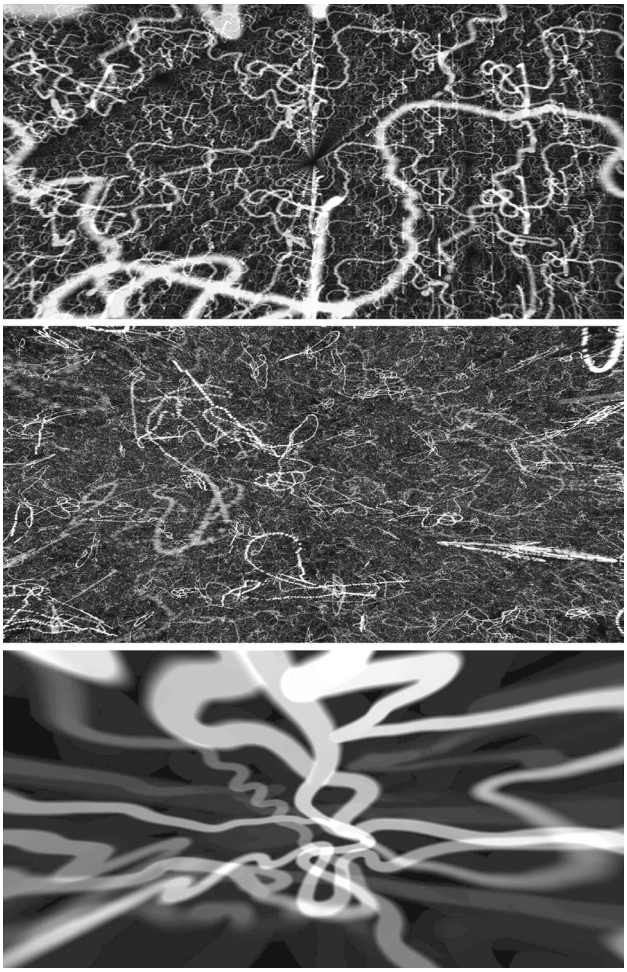


Fig. 6 Images of the immersive installation of the exhibit *"Regards dans les espaces de dimension 3"*. From *top* to *bottom* a curve in flat, hyperbolic and elliptic spaces

Ongoing and future work could go in many directions.

The above anti-aliasing is investigated to render radial and topological limit set of finitely generated Kleinian group. We are also currently extending the algorithm to other homogeneous space, such as solvable and nilpotent manifolds. For the four remaining Thurston geometries: the sphere $\mathbb{S}^3$, the universal covering $\widetilde{SL}_2$ of the modular group or the products $\mathbb{E}^1 \times \mathbb{H}^2$ and $\mathbb{E}^1 \times \mathbb{S}^2$ of the Euclidean line with the 2-hyperbolic space and the 2-sphere respectively, it is much simpler to adapt the presented algorithm. The issue is to deal only with the geodesics that project to real lines of the Euclidean space, as performed in [20].

Appendix: Polyhedral complexes that are not orbifolds

Here are examples of spaces that are possible to render with this new algorithm, but (presently) not with the previous ones.

A flat polyhedral complex that is not an orbifold Let P denote the regular planar octagon centered at 0. The angle of P are all equal to $3\pi/4$. The product of P with $[0, 1]$ is a polyhedron of the Euclidean space $\mathbb{E}^3$. As before, we glue the opposite faces with translations.

The polyhedral complex obtained is topologically the product of torus of genus 2 with a circle. However, it is not anymore diffeomorphic to it. The quotient identifies all the vertical edges and immerses them to a circle; in the polyhedral complexes, around this circle the angle is 6π. To be an orbifold, the angle must be a divisor of 2π.

A hyperbolic polyhedral complex that is not an orbifold The same construction can be done for any hyperbolic polyhedron. Then the dihedral angle between the faces is not necessarily a divisor of 2π and so the polyhedral complex is not necessarily a differentiable orbifold. Such a generality occurs for instance in [5], where the Einstein equation near a singularity is modeled by the geodesic flow on a (irregular) hyperbolic polyhedron, the faces of which are mirrors.

References

1. Amenta, N., Levy, S., Munzner, T., Phillips, M.: Geomview: a system for geometric visualization. In: Proceedings of the Eleventh Annual Symposium on Computational Geometry, SCG '95, pp. 412–413. ACM, New York (1995)
2. Arvo, J., Kirk, D.: An introduction to ray tracing. A Survey of Ray Tracing Acceleration Techniques, pp. 201–262. Academic Press Ltd., London (1989)
3. Boileau, M., Maillot, S., Porti, J.: Three-dimensional orbifolds and their geometric structures, volume 15 of Panoramas et Synthèses [Panoramas and Syntheses]. Société Mathématique de France, Paris (2003)
4. Burton, B.: Introducing regina, the 3-manifold topology software. Exp. Math. **13**(3), 267–272 (2004)
5. Damour, T., Henneaux, M., Nicolai, H.: Cosmological billiards. Class. Quant. Grav. **20**, R145–R200 (2003)
6. Francis, G.K., Goudeseune, C.M.A., Kaczmarski, H.J., Schaeffer, B.J., Sullivan, J.M.: Alice on the eightfold way: exploring curved spaces in an enclosed virtual reality theatre. In: Visualization and Mathematics III, pp. 305–315 (2003)
7. Gunn, C.: Advances in metric-neutral visualization. In: Skala, V., Hitzer, E. (eds.) GraVisMa 2010, pp. 17–26. Eurographics, http://gravisma.zcu.cz/GraVisMa-2010/GraVisMa-2010-proceedings.pdf (2010)
8. Gunn, C., Maxwell, D.L.: Not knot, http://www.geom.uiuc.edu/video/NotKnot/ (1991)
9. Gunn, C.: Discrete groups and visualization of three-dimensional manifolds. In: Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '93, pp. 255–262. ACM, New York (1993)
10. Haefliger, A.: Orbi-espaces. In: Sur les groupes hyperboliques d'après Mikhael Gromov (Bern, 1988), volume 83 of Progr. Math., pp. 203–213. Birkhäuser, Boston (1990)
11. Hanson, A.J., Munzner, T., Francis, G.: Interactive methods for visualizable geometry. Computer **27**(7), 73–83 (1994)
12. Hudson, R., Gunn, C., Francis, G.K., Sandin, D.J., DeFanti, T.A.: Mathenautics: using vr to visit 3-d manifolds. In: Proceedings

of the 1995 Symposium on Interactive 3D Graphics, I3D '95, pp. 167–170. ACM, New York (1995)
13. Munzner, T., Burchard, P.: Visualizing the structure of the world wide web in 3d hyperbolic space. In: Proceedings of the First Symposium on Virtual Reality Modeling Language, VRML '95, pp. 33–38. ACM, New York (1995)
14. Phillips, M., Gunn, C.: Visualizing hyperbolic space: unusual uses of 4×4 matrices. In: Proceedings of the 1992 Symposium on Interactive 3D Graphics, I3D '92, pp. 209–214. ACM, New York (1992)
15. Thurston, W.P.: The Geometry and Topology of Three-Manifolds. Princeton University, Princeton (1979)
16. University of Minnesota. The geometry center, http://www.geom.uiuc.edu/ (1994)
17. von Gagern, M., Richter-Gebert, J.: Hyperbolization of Euclidean Ornaments. Electron. J. Combin. **16**(2), 1–29 (2009)
18. Weeks, J.: The Shape of Space, http://www.geom.uiuc.edu/video/sos/ (1995)
19. Weeks, J.: Snappea, http://www.geometrygames.org/SnapPea/ (1999)
20. Weeks, J.: Real-time rendering in curved spaces. IEEE Comput. Graph. Appl. **22**(6), 90–99 (2002)
21. Weeks, J.: Real-time animation in hyperbolic, spherical, and product geometries. In: Prékopa, A., Molnár, E. (eds.) Non-Euclidean Geometries. Mathematics and Its Applications, vol. 581, pp. 287–305. Springer, Berlin (2006)
22. Weeks, J.R.: The shape of space. Pure and Applied Mathematics. Marcel Dekker, New York (2002)
23. Weismann, S., Gunn, C., Brinkmann, P., Hoffmann, T., Pinkall, U.: jreality: a java library for real-time interactive 3d graphics and audio. In: ACM Multimedia'09, pp. 927–928 (2009)

Pierre Berger is currently a CNRS researcher at Paris 13 university. A former student of Ecole Normale Supérieure de Paris (2001-2005), he did his PhD (2007) in hyperbolic dynamical systems in which he is a specialist. He is the author and the French coordinator of the exposition "Regards dans les espaces de dimension 3". He has experience in mathematics with emphasis on dynamical systems, geometry and topology.

Alex Laier currently serves as a professor at Universidade Federal Fluminense in the department of geometry. Ph.D. in applied mathematics from the Catholic University of Rio de Janeiro (2010), MA in mathematics from the Catholic University of Rio de Janeiro (2006) and BA in mathematics from the State University of Maringá (2002). He has experience in computer graphics, geometric modeling, data compression and has worked on developing projects in collaboration with Petrobras.

Luiz Velho is a full researcher/professor at IMPA - Instituto de Matematica Pura e Aplicada of CNPq , and the leading scientist of VISGRAF Laboratory. He received his BE in industrial design from ESDI/UERJ in 1979, a MS in computer graphics from the MIT/Media Lab in 1985, and a Ph.D. in computer science in 1994 from the University of Toronto under the Graphics and Vision groups. His experience in computer graphics spans the fields of modeling, rendering, imaging and animation.